Week 10 student worksheet — 75-minute guided lab

Student / team: _____ **Date:** _____

Evidence rule: Write `MEASURED`, `PREDICTED`, or `UNAVAILABLE` beside each claim. Never use zero or PASS as a placeholder for a missing tool result.

A. Freeze baseline (10 min) — Slide 17

1. RTL source file(s), module name, revision/hash:
2. Clock/active-edge and reset/active-polarity:
3. Input and output ports (including `valid_in` and `valid_out`, where applicable):
4. Count the three stored tap registers and write `y_out = d0 + 2*d1 + d2`.
5. State whether `y_out` is **registered or combinational** in the source you actually selected.
6. Source identity:
    - ☐ my verified Week 4 RTL
    - ☐ bundled teaching reference (not original)

B. Execute or trace Tcl (15 min) — Slides 13–18

Run `python3 run_lab.py --mode licensed --tool dc` from the package root, or `--mode offline` if tool access is intentionally not attempted.

- Status:
    - ☐ real DC reports generated
    - ☐ license/launcher error
    - ☐ offline-only
- Save actual tool log path (if attempted):
- Explain the following commands: `read_verilog`, `current_design`, `link`, `check_design`, `create_clock`, `compile`, `report_area`, `report_timing`, `write_file`.
- Why does checking out a `Design-Vision` license not prove that synthesis can run?

C. Netlist inspection (15 min) — Slides 9, 19

If real mapped.v exists: find top, cell names, flop pins, reset/enable logic, sign/adder/mux optimizations; copy two actual instance names.

If not: inspect `examples/illustrative_enable_stage.v`; it uses made-up *tutorial modules* and is **not** a T18 mapped netlist. Identify the DFF, feedback MUX, selection signal and hold path. Explain why an integrated-enable FF would be another possible implementation.

- Prediction vs observation:
- Evidence label:

D. Area (10 min) — Slide 20

- Write `A_cells = Σ_i (N_i × A_i)` and define `N_i` and `A_i`.
- Where would a real library cell-area unit be found?

- Why is `mapped cell area ≠ core area ≠ die area`?
- Measured area available?
  - ☐ yes (cite report)
  - ☐ no (write UNAVAILABLE)

E. Timing (10 min) — Slides 11–12, 21

- For the **illustrative** setup budget: T = 10, tCQ = 0.7, logic+net = 6.5, setup = 1.0, uncertainty = 0.3 ns. Compute slack.
- If period alone becomes 8 ns but all illustrative delays are artificially fixed, compute slack again. Why is this **not** a real 8 ns Design Compiler result?
- For actual `setup.rpt` and `hold.rpt`, identify startpoint, endpoint, arrival, required, slack and whether each reported path was constrained. Explain max-delay vs min-delay checks.
- Compare reported setup and hold evidence; does positive setup slack guarantee hold closure?

F. Constraint coverage and controlled comparison (10 min) — Slides 22–23

- Does the clock target the real RTL clock port?
- Are *all normal data input* and output ports included in the assumptions?
- How should `rst_n` (an asynchronous reset) be handled responsibly?
- Review `timing_coverage.rpt` (`check_timing`) and `violations.rpt`; what remains unproven even if `check_timing` prints no errors?
- What warnings could invalidate an otherwise apparently successful run?
- Compare the actual 10 ns and 8 ns DC area/slack **only if both runs completed**; otherwise predict possible effects without numbers.

Exit/hand-in

- ☐ frozen source/manifest
- ☐ annotated Tcl + SDC assumptions
- ☐ offline notes or real logs
- ☐ reference model trace (if bundled design)
- ☐ report/netlist *only if real*

In one sentence: Why does positive pre-layout slack not prove post-route timing closure?